

Practical 1

A - Aws creation

B - Building and Training a Machine Learning Model for Telecom Tower Failure Prediction

train.py

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

import joblib

import json


print("Loading Dataset...")

df = pd.read_csv("Telecom_Tower_Failure_Dataset_10000-1.csv")

X =
df[['Tower_ID','Temperature_C','Battery_Voltage','Power_Consumption_W','Signal_Strength_Percentage',
    'Fan_Speed_RPM','Humidity_Percent','Traffic_Load','Tower_Age_Years']]

y = df['Failure_Within_48Hrs']


X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.25,random_state=42)


print("Training Model...")

model=RandomForestClassifier(n_estimators=100,random_state=42)

model.fit(X_train,y_train)

accuracy=model.score(X_test,y_test)

print("Accuracy :",accuracy)


joblib.dump(model,"telecom_tower_model.pkl")


metrics={"accuracy":accuracy}

with open("metrics.json","w") as f:

    json.dump(metrics,f,indent=4)

print("Training Completed Successfully")
```

predict.py

```
import joblib

import pandas as pd

model = joblib.load("telecom_tower_model.pkl")

new_data = pd.DataFrame({
    "Tower_ID": [1001],
    "Temperature_C": [55],
    "Battery_Voltage": [11.2],
    "Power_Consumption_W": [420],
    "Signal_Strength_Percent": [82],
    "Fan_Speed_RPM": [2500],
    "Humidity_Percent": [60],
    "Traffic_Load": [75],
    "Tower_Age_Years": [8]
})

prediction = model.predict(new_data)

if prediction[0] == 1:
    print("Hardware Failure Predicted")
else:
    print("Tower is Healthy")
```

AWS:

Go to AWS and create bucket and upload files

Your bucket should look like

telecom-mlops-bucket

└─ data folder

│ └─ tower.csv

└─ models folder

└─ telecom_tower_model.pkl

Practical 2: CI/CD Pipeline for Telecom Tower Failure Prediction using AWS CodePipeline

Create git repo

Upload all files

Commit

buildspec.yml

version: 0.2

phases:

install:

runtime-versions:

python: 3.11

commands:

- pip install -r requirements.txt

build:

commands:

- python train.py

artifacts:

files:

- model.pkl

Requirments.txt

pandas

numpy

scikit-learn

joblib

Create s3 bucket: telecom-mlops-model-storage

Create IAM role

1. Select:

- **Trusted Entity:** AWS Service
- **Use Case:** CodeBuild

2. Attach the following policies:

- AmazonS3FullAccess
- AWSCodeBuildDeveloperAccess
- CloudWatchLogsFullAccess

Create AWS codebuild project

Procedure

1. Open **AWS CodeBuild**, Click **Create Build Project**.

Project Name: TelecomModelTraining

2. Source Provider: GitHub
3. Connect your **Telecom-MLOps** repository.
4. Environment:
 - Operating System: Ubuntu
 - Runtime: Managed Image
 - Language: Python
5. Service Role: CodeBuildTelecomRole
7. Build Specification: Use buildspec.yml
8. Click **Create Build Project**.

Test the build project: open the project and start build (build history – latest build – view logs)

Create aws codepipeline

Procedure

1. Open **AWS CodePipeline**. Click **Create Pipeline**.
Pipeline Name: TelecomMLOpsPipeline
3. Source Stage:
 - Provider: GitHub Version 2
 - Repository: Telecom-MLOps
 - Branch: main
4. Build Stage:
 - Provider: AWS CodeBuild
 - Project: TelecomModelTraining
5. Deployment Stage: Skip Deployment
6. Click **Create Pipeline**.

Trigger the automatic training

Open train.py.

Change:

n_estimators = 100 to n_estimators = 150

Save the file.

Commit the changes.

Push the updated code to GitHub.

verify the execution that s3 bucket has latest model.pkl file

Practical 3: Apache Jmeter

app.py (pip install flask)

```
from flask import Flask, request, jsonify
```

```
import pandas as pd
```

```
import joblib
```

```
app = Flask(__name__)
```

```
model = joblib.load("telecom_tower_model.pkl")
```

```
@app.route("/")
```

```
def home():
```

```
    return "Telecom Tower Prediction API is Running Successfully!"
```

```
@app.route("/predict", methods=["POST"])
```

```
def predict():
```

```
    data = request.get_json()
```

```
    df = pd.DataFrame([data])
```

```
    prediction = model.predict(df)
```

```
    return jsonify({
```

```
        "prediction": int(prediction[0])
```

```
    })
```

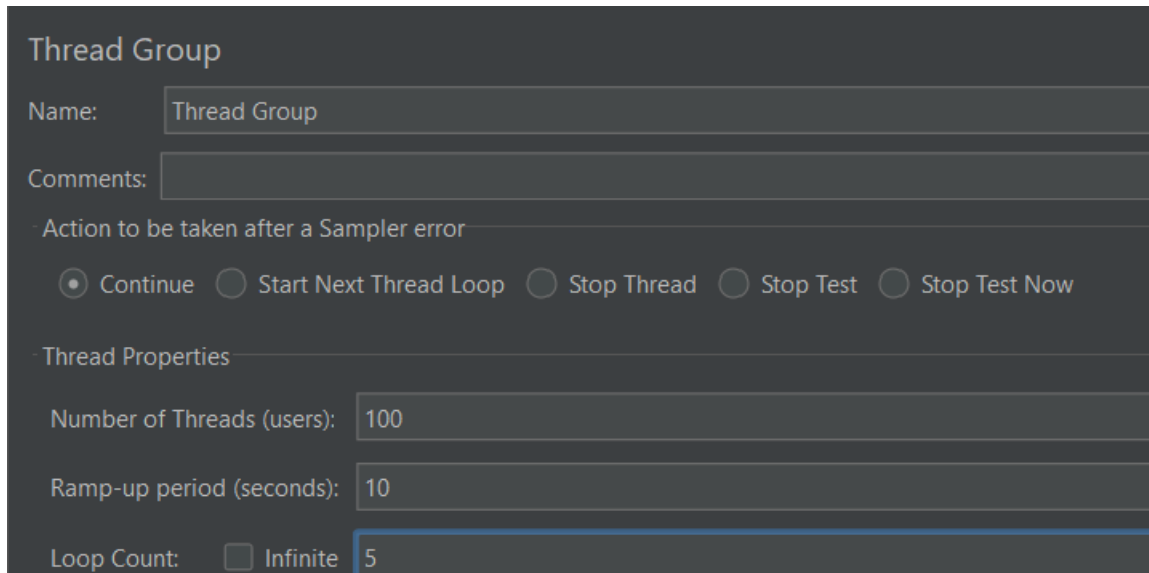
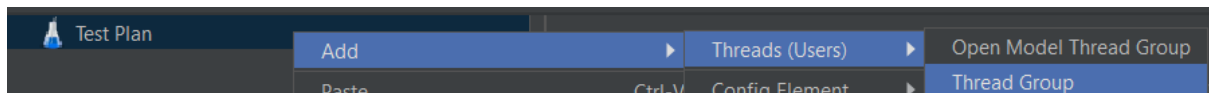
```
app.run(debug=True)
```

run the file and click the link to see the web page

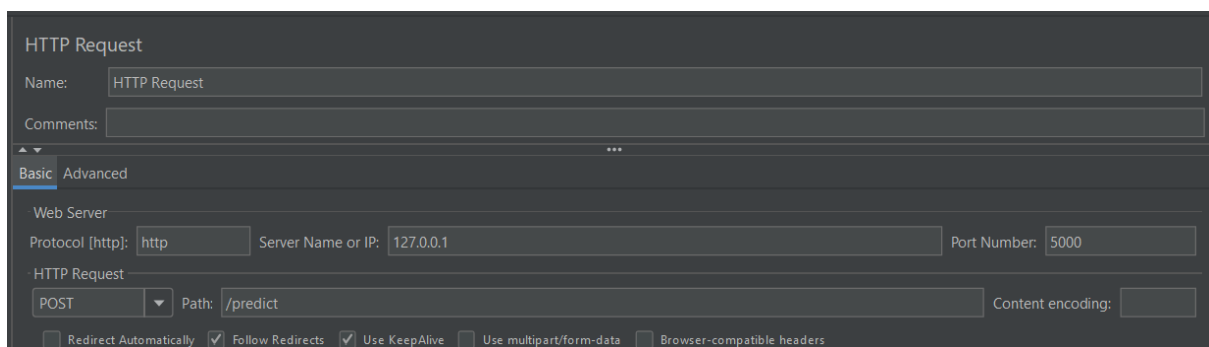
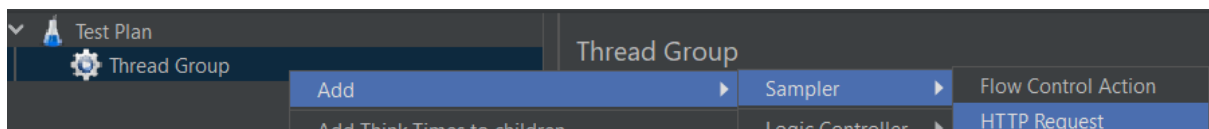
open jmeter – extract the zip file and run jmeter.bat file

Create a test plan and name

Add thread group: 100 users, 10 seconds, 5 loops

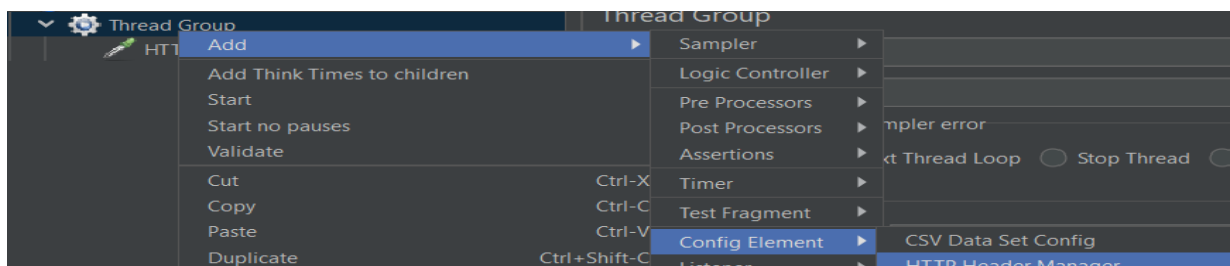


Add HTTP Request: protocol - http, server - 127.0.0.1, Port - 5000, Method - POST, Path - /predict



Click Add

Add Http header manager



Name: Content-Type, Value – application/json

HTTP Header Manager

Name:

Comments:

Headers Stored in the Header Manager

Name:	Value
Content-Type	application/json

Add JSON Data: **give same features as your predict.py file used in other prac**

Http request (see below middle tab)→ Body Data and type json data

```
{  
  "Tower_ID": 1001,  
  "Temperature_C": 55,  
  "Battery_Voltage": 11.2,  
  "Power_Consumption_W": 420,  
  "Signal_Strength_Percent": 82,  
  "Fan_Speed_RPM": 2500,  
  "Humidity_Percent": 60,  
  "Traffic_Load": 75,  
  "Tower_Age_Years": 8  
}
```

HTTP Request

Name:

Comments:

Basic Advanced

Web Server

Protocol [http]: Server Name or IP:

HTTP Request

Path:

☐ Redirect Automatically ☒ Follow Redirects ☒ Use Keep-Alive

Parameters Body Data Files Upload

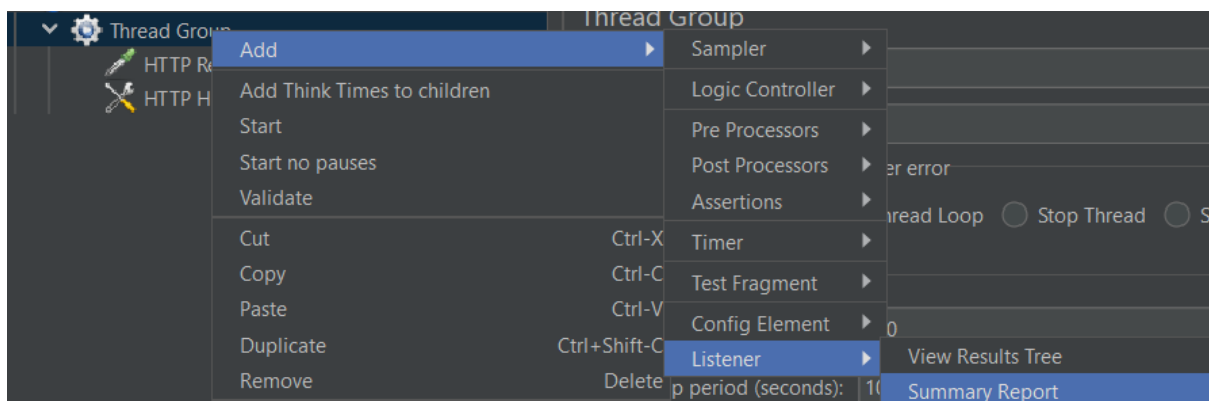
```

1 {
2   "Tower_ID": 1001,
3   "Temperature_C": 55,
4   "Battery_Voltage": 11.2,
5   "Power_Consumption_W": 420,
6   "Signal_Strength_Percent": 82,
7   "Fan_Speed_RPM": 2500,
8   "Humidity_Percent": 60,
9   "Traffic_Load": 75,
10  "Tower_Age_Years": 8
11 }

```

Run your app.py file and keep browser open – flask application then

Add a Summary Report and View results tree



Click the play button to see the result in summary table

Summary Report

Name:

Comments:

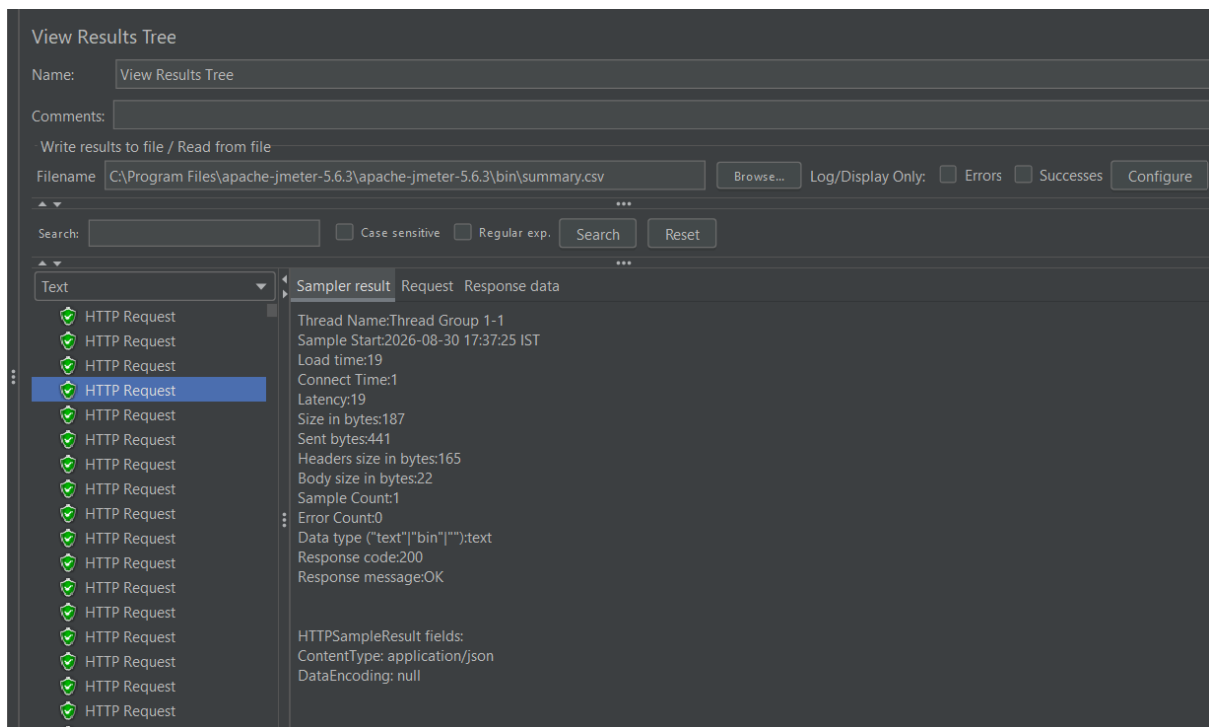
Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

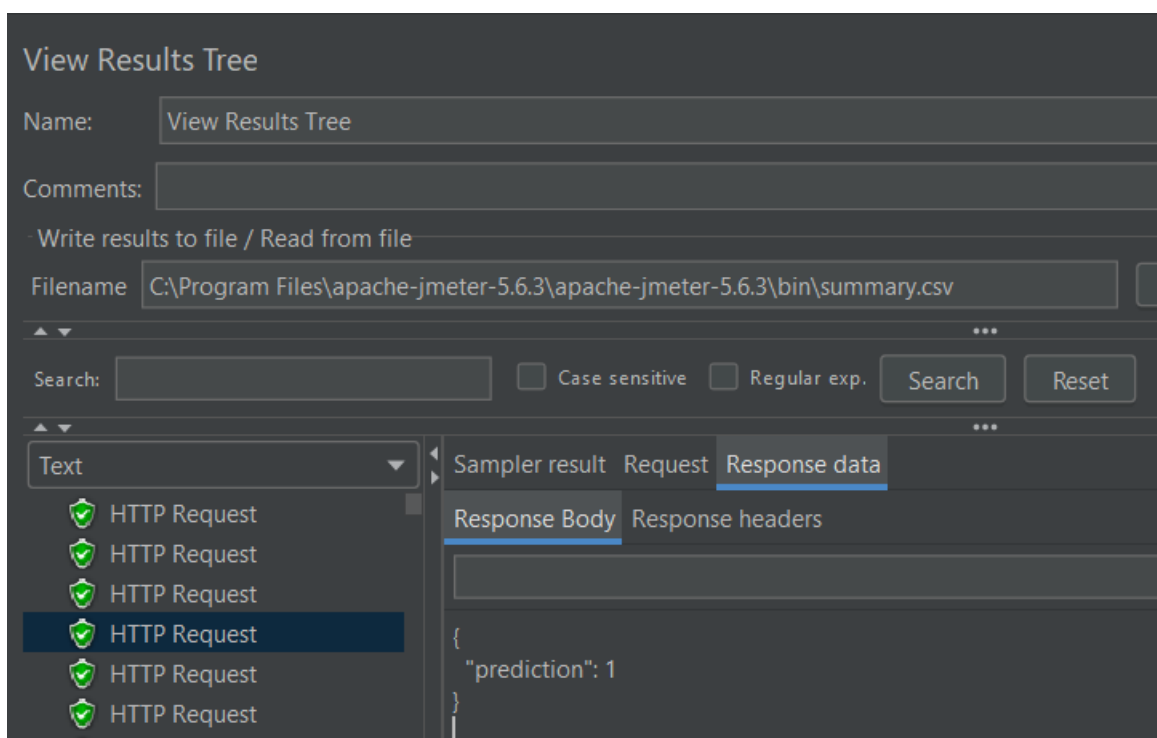
Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/...	Sent KB/sec	Avg. Bytes
HTTP Request	500	8	5	70	6.63	100.00%	50.3/sec	1085.72	21.65	22111.0
TOTAL	500	8	5	70	6.63	100.00%	50.3/sec	1085.72	21.65	22111.0

And now view the results tree click Run

Click any of one http request you can see sampler result, request, response data



Click on response data tab to view the final prediction



Final prediction will be 0 or 1

Practical 4: Github commands

```
git --version (to see git is installed or not)
git init      (initialise empty repo hidden .git folder created)
git config --global user.name "your username"
git config --global user.email "your email"
git config --list
git status
git add .
git status
git commit -m "First commit"
git remote add origin "https://github.com/your username/mlops_lab.git"
git remote -v
git branch -M main
git push -u origin main
git add train.py
git commit -m "changes"
git push
```

Practical 5: MLflow UI

monitor_model.py

```
import pandas as pd
import joblib
import mlflow

from sklearn.metrics import accuracy_score

model = joblib.load("telecom_tower_model.pkl")
print("Model loaded successfully!")

new_data = pd.read_excel("new_tower_telemetry.xlsx")
print("New production data loaded successfully!")

features = [
    'Tower_ID',
    'Temperature_C',
```

```

'Battery_Voltage',
'Power_Consumption_W',
'Signal_Strength_Percent',
'Fan_Speed_RPM',
'Humidity_Percent',
'Traffic_Load',
'Tower_Age_Years'
]
X_new = new_data[features]
y_new = new_data["Failure_Within_48Hrs"]
predictions = model.predict(X_new)
accuracy = accuracy_score(
    y_new,
    predictions
)
print("-----")
print("MODEL DRIFT MONITORING")
print("-----")
print(
    "Production Accuracy:",
    round(accuracy, 2)
)
mlflow.set_experiment(
    "Monitor Model"
)
with mlflow.start_run():
    mlflow.log_param(
        "model",
        "Random Forest"
    )

```

```
mlflow.log_param(
    "n_estimators",
    100
)
mlflow.log_metric(
    "production_accuracy",
    accuracy
)
print("Results logged successfully to MLflow")

threshold = 0.80

if accuracy < threshold:
    print()
    print("WARNING: Model performance has degraded!")
    print("Model retraining is recommended.")
else:
    print()
    print("Model performance is stable.")
```

run this code in cmd and then

pip install mlflow ui

mlflow ui

click on the link and check the dashboard

Inside the run, you should find:

Parameters

model = Random Forest

n_estimators = 100

Metric

production_accuracy = 0.xx

This demonstrates that **MLflow is tracking the model's production performance.**